

Capítulo 3: Tipos de Datos en Python

Cuaderno de Resumen Técnico y Guía de Referencia

1. CONCEPTOS FUNDAMENTALES SOBRE LOS DATOS

- **Definición Interna:** En memoria, los programas y datos son secuencias de bits.
- **Todo es un Objeto:** En Python, cualquier valor (número, cadena, función) se maneja como un objeto en memoria RAM.
- **Atributos de un Objeto:** Todo objeto posee mínimamente:
 - **Tipo:** Define el conjunto de valores y operaciones permitidas.
 - **Identificador único (ID):** Dirección de memoria para distinguirlo.
 - **Valor:** Dato concreto almacenado.

2. TIPOS DE DATOS BÁSICOS Y CLASIFICACIÓN

Nombre	Tipo (type)	Ejemplos	Mutabilidad
Booleano	<code>bool</code>	<code>True</code> , <code>False</code>	Inmutable
Entero	<code>int</code>	<code>21</code> , <code>34500</code> , <code>34_500</code>	Inmutable
Flotante	<code>float</code>	<code>3.14</code> , <code>1.5e3</code>	Inmutable
Complejo	<code>complex</code>	<code>2j</code> , <code>3+5j</code>	Inmutable
Cadena	<code>str</code>	<code>'tfn'</code> , <code>'''tenerife'''</code>	Inmutable
Tupla	<code>tuple</code>	<code>(1, 3, 5)</code>	Inmutable
Lista	<code>list</code>	<code>['Chrome', 'Firefox']</code>	Mutable
Conjunto	<code>set</code>	<code>set([2, 4, 6])</code>	Mutable
Diccionario	<code>dict</code>	<code>{'Chrome': 'v79'}</code>	Mutable

Inmutabilidad

No se puede modificar el objeto directamente en memoria. Asignar un nuevo valor hace que la variable apunte a una nueva dirección.

Mutabilidad

El contenido del objeto puede ser alterado *in-situ* sin cambiar la dirección de memoria original del objeto contenedor.

3. VARIABLES, CONSTANTES Y ASIGNACIÓN

- **Reglas de Nombres:** Letras (a-z, A-Z), dígitos y guiones bajos (`_`). Deben iniciar con letra o guión bajo, jamás con un dígito. Son sensibles a mayúsculas/minúsculas (*case-sensitive*).
- **Palabras Reservadas:** No se pueden usar identificadores reservados por la sintaxis de Python (ej. `if` , `for` , `class` , `True` , `pass`).
- **Convención PEP 8:** Usar `snake_case` en minúsculas para variables (`num_songs`) y `UPPER_CASE` para constantes (`LIGHT_SPEED = 300_000`).
- **Asignación Múltiple:** Es posible asignar un valor a varias variables simultáneamente (ej. `a = b = c = 0`).

4. NÚMEROS Y OPERACIONES ARITMÉTICAS

Tipos Numéricos Principales

- **Enteros (`int`)**: Sin límite de tamaño práctico. Se pueden usar `_` como separadores (ej. `1_000_000`).
- **Flotantes (`float`)**: Números decimales bajo estándar IEEE 754. Tienen límite de precisión. Usar `round(val, n)` para mitigar imprecisiones.
- **Booleanos (`bool`)**: Subtipo de entero con valores `True` (1) y `False` (0).

Operadores Aritméticos y Prioridad de Evaluación

Prioridad	Operador	Descripción	Ejemplo	Resultado
1 (Alta)	()	Paréntesis de aglutinación	<code>(2 + 4) * 3</code>	18
2	<code>**</code>	Exponenciación / Potencia	<code>2 ** 4</code>	16
3	<code>+a</code> , <code>-a</code>	Signo unario (positivo/negativo)	<code>-99</code>	-99
4	<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Multiplicación, División flotante, División entera, Módulo	<code>9 / 2</code> <code>9 // 2</code>	4.5 4
5 (Baja)	<code>+</code> , <code>-</code>	Suma y Resta	<code>3 + 9</code>	12

5. CADENAS DE TEXTO (`STR`)

Secuencias inmutables de caracteres Unicode. Soporta comillas simples (`'`), dobles (`"`) y triples (`'''`) para multilínea.

- **Secuencias de Escape**: `\n` (salto de línea), `\t` (tabulador), `\'` , `\"` , `\\` . Las cadenas en bruto (`r'...'`) ignoran el escape.
- **Indexación y Slicing**: `cadena[0]` accede al primer carácter, `cadena[-1]` al último. Slicing: `cadena[inicio:fin:paso]` .
- **Métodos Clave**:
 - **Limpieza**: `.strip()` , `.lstrip()` , `.rstrip()`
 - **Transformación**: `.upper()` , `.lower()` , `.capitalize()` , `.title()`
 - **Búsqueda/Subcadenas**: `.find()` , `.count()` , `.startswith()` , `.split()`

```
# Formateo mediante f-strings (Python 3.6+) nombre = "Elon" pi = 3.14159 print(f"Hola {nombre}! El valor de pi es {pi}")
```

6. FUNCIONES INTEGRADAS CLAVE (BUILT-IN)

- `type(x)` : Retorna el tipo de dato.
- `isinstance(x, tipo)` : Verifica el tipo del objeto.
- `id(x)` : Retorna el identificador en memoria.

- `len(x)` : Devuelve la longitud del objeto.
- `input("msg")` : Lee entrada de texto por teclado.
- `dir(x)` : Muestra atributos y métodos disponibles.